edikit: A Python Toolkit for Reproducible Construction and Auditing of National Social Accounting Matrices

Kodzovi Abalo
World Bank
ORCID 0000-0002-7567-2181

Abstract

`edikit` is a small Python library for building and auditing national social accounting matrices (SAMs) from official statistics under a discipline not yet systematically embedded in commonly distributed SAM construction workflows: accounting identities are enforced as executable checks at ingestion and at subsequent pipeline boundaries, classification is never inferred silently from labels or code conventions, every balancing adjustment is logged at the cell where it acts, and every run writes a validation report that constitutes the record. The toolkit was developed and validated across five countries and three exercise types. A cell-level reconstruction and audit of the published 2019 South African SAM reproduced the production and commodity structure exactly across all 61 activities and every final-demand category, and a structure-and-macro-controls audit covered the IFPRI 2019 Kenya SAM. Benchmark-free generation then produced balanced macro SAMs for the Netherlands, Austria, and Poland from the Eurostat dissemination API—three HTTP requests per country, one command, with confidentiality suppression detected and quantified where present. Users enter through three documented recipes—replicate, audit, and build—and the package has one required external dependency, `openpyxl`.

Keywords: social accounting matrix; supply and use tables; national accounts; reproducibility; RAS balancing; data provenance; Eurostat; Python.

1 Overview

1.1 Introduction

Social accounting matrices are the standard database for multiplier and computable general equilibrium analysis, yet their construction remains largely artisanal: heterogeneous official tables are reconciled through spreadsheet operations and expert judgments that are rarely documented at the level where they act, so published SAMs can seldom be reproduced, audited, or updated by anyone other than their authors. The companion methods paper (1) develops this diagnosis through a detailed cell-level replication audit of the unusually well documented 2019 South African SAM of van Seventer and Davies (2), and proposes a remedy: cell-level provenance and machine-readable construction rules published alongside every SAM. `edikit` is the reference implementation of that proposal, released so that the remedy is a working artefact rather than an exhortation.

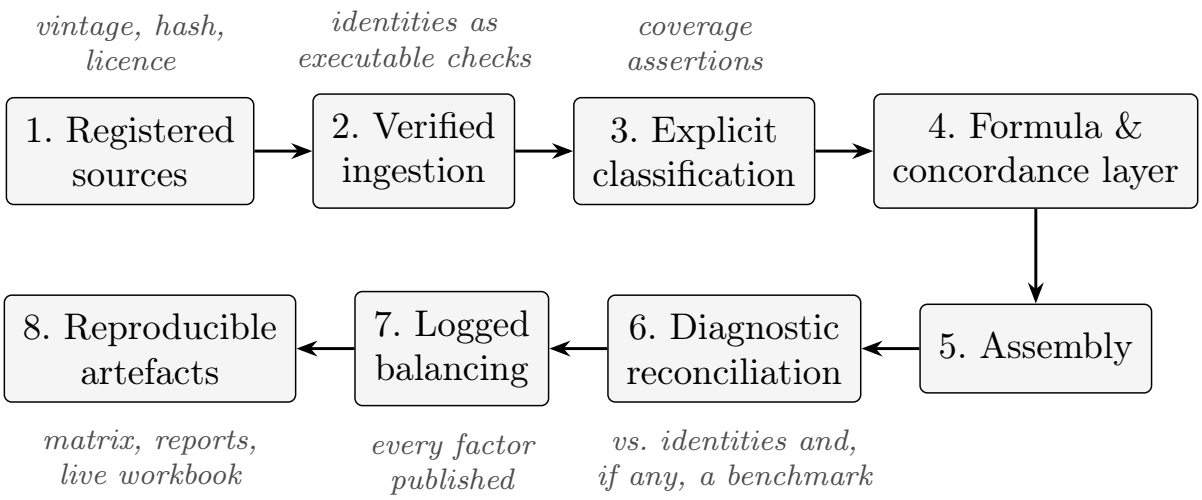

Figure 1: The workflow `edikit` implements, from registered sources to reproducible artefacts.

The toolkit's design reduces to four commitments. Every source file is registered with vintage and hash before use. Accounting identities—for each product, supply at purchasers' prices equals basic-price supply plus taxes plus margins; for each industry, output equals intermediates plus value added; for the matrix, every account's receipts equal its payments—are enforced as executable checks the moment data are read, not asserted afterwards. Classification is always explicit: account codes, labels, and header positions are treated as claims to verify, because the audit found them wrong in even the best-documented matrix in its class. And adjustment is never silent: the balancing routine returns the multiplicative factor of every cell it touches, and refuses inputs it cannot handle transparently.

Existing open tooling in the input–output ecosystem concentrates primarily on consuming and analysing completed IO and MRIO databases—`pymrio` (3) is the leading example, providing parsers, standardisation, and footprint analytics over compiled databases such as EXIOBASE and WIOD. `edikit` addresses an earlier, complementary stage of the pipeline: an integrated workflow for *constructing and auditing* a national SAM against its underlying official sources. Its distinguishing features are not new SAM mathematics—RAS, concordances, and the accounting identities are long established—but the integration of executable accounting checks at ingestion, explicit and complete account mappings, source-level provenance, and cell-specific diagnostics for every balancing adjustment into one testable source-to-artefact workflow.

1.2 Implementation and Architecture

Figure 1 shows the workflow the library implements; the same stages serve construction and audit, an audit adding a comparison target at the reconciliation stage.

`edikit` is pure Python (about 1,500 lines in the implemented core), organised in four modules with one external dependency (`openpyxl`, for Excel I/O):

- `edikit.data` — readers for the source families a SAM build encounters: supply–use workbooks (recomputing every embedded total and enforcing the layout's identities), distributed SAM matrices, central-bank time-series bulk files with a small evaluator for

signed series formulas, Eurostat JSON-stat responses (decoding the dissemination API's linear index into code-keyed cells), and concordances with completeness validation.

- `edikit.model` — the account taxonomy with coverage assertions (every account classified, nothing phantom; classification is never inferred silently from labels or code conventions—any prefix rules must be explicitly declared, reviewable, and complete, and unmatched accounts raise) and RAS balancing that requires a non-negative seed, verifies target consistency, reports convergence, and returns per-cell adjustment factors as a first-class result.
- `edikit.report` — generation of Excel workbooks whose totals and balance checks are live formulas rather than pasted numbers, so a reader can interrogate the artefact in the tool they already use.
- `edikit.pipeline` — end-to-end routines: `eurostat_sam` (fetch, generate, balance, and report a macro SAM for countries the ESA 2010 transmission datasets cover, including coverage-based classification-tiling assertions and suppressed-detail handling) and `audit` (structural audit of any distributed SAM, plus macro-aggregate comparison against published national-accounts controls).

Table 1 lists the principal public entry points.

Module	Key callables	Role
`data.sut`	`read_supply_table`, `read_use_table`	SUT ingestion with identity checks
`data.sam`	`read_labeled_matrix`	distributed SAM workbooks
`data.kbp`	`read_kbp_zips`, `evaluate_formula`	central-bank series and signed formulas
`data.eurostat`	`read_jsonstat`, `EurostatTable`	dissemination-API responses
`data.concordance`	`Concordance`	mappings with completeness validation
`model.accounts`	`AccountSet`, `AccountKind`	explicit taxonomy, coverage assertions
`model.balancing`	`ras`, `gras`, `BalanceResult`	RAS and GRAS with per-cell factor diagnostics
`report.workbook`	`add_matrix_sheet`, `add_table_sheet`	live-formula Excel artefacts
`pipeline.eurostat_sam`	`fetch`, `generate`, `balance`	macro-SAM generation (ESA countries)
`pipeline.audit`	`audit`, `read_kinds`	structural and macro-controls auditing

Table 1: Principal public entry points by module.

Users enter through three documented *recipes*, thin scripts over the library: `replicate/` regenerates every number in the companion paper; `audit/` audits a published SAM (`python audit_sam.py --sam matrix.csv --kinds kinds.csv --controls controls.csv`); `build/` generates and balances a macro SAM for a country-year (`python build_sam.py --country AT --year 2019`). Every pipeline run writes a markdown validation report recording each identity check, coverage diagnostic, and account residual, and—when balancing is invoked—convergence, flipped cells, and the adjustment-factor range, with the factor of every individual

cell in the balanced CSV; the reports, not the console, are the record. The recipes are thin by design: the same functionality is available through a short library workflow, for example:

```
from edikit.pipeline import eurostat_sam as es

paths = es.fetch("AT", 2019, "data/")     # three cached API requests
res   = es.generate("AT", 2019, paths)    # identities checked; residuals attributed
bal, flipped = es.balance(res)            # RAS; every factor in bal.factors
es.write_report(res, "AT_2019_generation.md",
                balance=bal, flipped=flipped)
es.write_balanced_csv(bal, "AT_2019_macro_sam_balanced.csv")
```

The Dutch case runs entirely offline from three registered Eurostat responses committed in the repository, which is the recommended first run (the repository README's "reviewer pathway"). Its validation report reads, in excerpt:

```
Output and VA identities: 0 findings at EUR 1m tolerance
Commodity balance closure: 65/65 products within EUR 1m
GDP at basic prices (generated): EUR 724,960m
lab residual: -4,193 EUR m (-0.578% of GDP)
row residual: -10,830 EUR m (-1.494% of GDP)
RAS converged in 418 iterations; max residual EUR 0.009m
Adjustment factors: 70 cells in [0.9832, 1.0282]
```

1.3 Quality Control

Three layers. First, a `pytest` suite (38 tests at the tagged release) in which the accounting identities are the test cases, covering the readers, the classification and tiling machinery, RAS and GRAS diagnostics, the audit pipeline, the cross-producer reconciliation reporter, and end-to-end generation and balancing on a synthetic two-industry economy whose identities close exactly. Every test runs from inputs shipped in the archive, so a reviewer sees `38 passed`. Line coverage is 93 per cent overall (84–100 per cent per implemented module; `pytest toolkit/tests --cov=toolkit/src/edikit`); this measures which code paths are exercised, while the identity-based and end-to-end tests carry the evidence of domain correctness. A continuous-integration workflow runs the suite on Linux, macOS, and Windows across Python versions 3.11–3.13 on every push. Beyond the five validation countries, a committed coverage sweep (`recipes/build/sweep.py`) runs the build pipeline across 37 candidate countries for 2019 and classifies every outcome on the toolkit's own terms: 11 pass every core check (zero identity findings, full commodity closure, no cross-table differences, import identity within tolerance, no suppression, converged balancing, worst residual within 5 per cent of GDP); 9 generate and balance with documented caveats; 7 carry large residuals or extreme factors and require review; 2 generate but cannot be balanced from one-sided sector accounts; and 8 country-years are not covered by the datasets. The archived Zenodo deposit is the snapshot of the tagged release this paper describes, and the reviewer pathway's expected console output is stated verbatim in the repository README, so a successful installation is verifiable in minutes. Second, run-time

Country (2019)	Exercise	Outcome
South Africa	cell-level reconstruction, audited against the published UNU-WIDER benchmark	Production and commodity structure exact on all 61 activities and every final-demand category; 43 of 45 computable macro cells exact. The two exceptions both reflect one unrecorded R596 million reconciling adjustment applied symmetrically; the benchmark's single unpublished input was priced separately.
Kenya	structural and macro-controls audit of the IFPRI Nexus SAM	structure verified; matrix balanced to its distribution's rounding grain; production side within 1% of published controls
Netherlands	benchmark-free generation and balancing	zero identity findings (65/65 industries and products); all residuals attributed, each $\leq$1.5% of GDP; balanced to <EUR 0.01m at full precision with factors in $[0.9832, 1.0282]$
Austria	transferability: the same generator rerun unchanged on a new country	zero identity findings; residuals $\leq$0.6% of GDP
Poland	suppression stress test	withheld cells detected (leaf detail covers 96.7% of gross output); macro accounts anchored to published totals; residuals $\leq$1.1% of GDP

Table 2: Validation across five countries and three exercise types.

assertions in every pipeline: coverage assertions on classification, tiling assertions on category-list partitions, target-consistency checks in balancing. These assertions have each caught real errors during development—a survey parser that silently dropped an industry with 6,202 observations, and a double-count from Eurostat's mixed-granularity category lists—which the companion paper reports as evidence that the checks bind. Third, validation at scale across five countries and three exercise types, summarised in Table 2; each run's full validation report ships with the repository, and the Polish case demonstrates the principal failure mode handled transparently—confidentiality suppression is detected by the tiling assertions, quantified per aggregate, and reported rather than absorbed.

2 Availability

2.1 Operating system

Pure Python; the test suite runs in continuous integration on Linux, macOS, and Windows.

2.2 Programming language

Python $\geq$ 3.11.

2.3 Additional system requirements

None; memory and disk requirements are trivial (national SUT and sector-accounts inputs are megabytes).

2.4 Dependencies

`openpyxl`. Optional: `certifi` (TLS trust store for API fetches on platforms without system certificates); `pytest` and `pytest-cov` (development).

2.5 Software location

Code repository: GitHub, `https://github.com/kodzovee9/tekmeris` (toolkit under `toolkit/`); licence Apache-2.0; version 0.2.1, July 2026.

Archive: Zenodo, DOI 10.5281/zenodo.21442608 (version v0.2.1; concept DOI for all versions: 10.5281/zenodo.21419195); licence Apache-2.0.

2.6 List of contributors

Kodzovi Abalo (design, development, validation).

2.7 Language

English.

3 Reuse Potential

Three reuse paths correspond to the three recipes. Researchers can generate a validated, balanced macro SAM with one command and no manual data step for the countries the Eurostat datasets cover—the committed coverage table documents exactly which: for 2019, 11 of 37 candidates pass every core accounting check, a further 9 generate and balance with documented caveats, and every remaining case is classified and recorded. The same command run across countries gives comparably constructed matrices with per-country validation reports—including, where publication practice withholds cells, an explicit statement of what was suppressed. Analysts holding any distributed SAM can run the structural audit unmodified (two CSV sidecars—an account-classification table and published controls— enable the macro comparison), which makes routine due diligence on third-party SAMs practical for model users who did not build the matrix they depend on. And compilers working outside the harmonised universe can adapt the South African pipeline pattern documented in the recipes: registered sources, identity-checked ingestion, explicit concordances, formula tables, and logged balancing, reusing the library wholesale and replacing only country-specific configuration.

Reusers should know the current limitations plainly. Version 0.2.1 implements classical RAS and GRAS (4)—the latter negative-tolerant, so matrices carrying negative cells can be balanced directly. Standard RAS requires a non-negative seed. The Eurostat pipeline applies an explicitly reported sign-normalisation rule to the limited negative flows it encounters, and refuses inputs it cannot handle that way; matrices with general negative entries should use GRAS instead. The toolkit does not implement general cross-entropy balancing against an arbitrary prior (5), which matters most where margins are partial or additional constraints are imposed. The automatic Eurostat workflow produces a macro SAM, not a fully disaggregated multi-sector

and household SAM. Adaptation outside the Eurostat universe requires country-specific source readers and mappings. Statistical confidentiality can prevent full cell-level reconstruction—the toolkit detects and quantifies suppression, but cannot undo it. And an accounting-consistent output is not necessarily an economically preferred allocation where multiple admissible mappings exist; the toolkit's contribution is to make the chosen mapping visible, not to certify it unique.

Natural extensions, deliberately scoped out of version 0.2.1, include cross-entropy balancing against a prior under partial margin information (5), full multi-sector (rather than macro) generation from the 65-industry ESA core, and survey-driven household disaggregation following the companion paper's South African scripts. User support, bug reports, and feature requests are managed through the repository's GitHub Issues page; contributions are welcome, and the test suite and the identity-as-assertion discipline define the contract new code must meet.

Data Accessibility Statement

The versioned software, redistributable sample inputs, validation reports, and generated outputs are archived on Zenodo at doi:10.5281/zenodo.21442608. Third-party inputs whose licences do not permit redistribution are not included; for each restricted input, the repository's `SOURCES.md` records the publisher, source location, access date, licence or access conditions, and SHA-256 checksum required to verify a locally obtained copy.

Generative AI Use

Generative AI (Anthropic's Claude) was used for code generation and diagnosis. All generated code, mappings, and outputs were reviewed by the author and accepted only after the deterministic accounting checks and tests described above were satisfied. The author assumes full responsibility for the software.

Author Roles

Kodzovi Abalo: conceptualisation, software design and development, validation, writing.

Acknowledgements

The toolkit's validation relied on data published or distributed by Statistics South Africa, the South African Reserve Bank, the Kenya National Bureau of Statistics, IFPRI, UNU-WIDER, and Eurostat; the unusually thorough documentation of the UNU-WIDER 2019 South African SAM (2) is what made cell-level validation possible.

Disclaimer

The views expressed here are those of the author and do not necessarily reflect the views of the World Bank, its Executive Directors, or the governments they represent.

Funding Statement

No dedicated funding was received for the development of the software or the preparation of this manuscript.

Competing Interests

The author declares no competing interests.

References

1. Abalo K. From supply and use tables to reproducible social accounting matrices: a transparent construction and audit framework [preprint]; 2026. doi:10.5281/zenodo.21426100 (concept DOI; resolves to the latest version).

2. van Seventer D, Davies R. A 2019 social accounting matrix for South Africa with occupational and capital stock detail. Helsinki: UNU-WIDER; 2023. UNU-WIDER Technical Note 2023/1. doi:10.35188/UNU-WIDER/WTN/2023-1.

3. Stadler K. Pymrio – a Python based multi-regional input-output analysis toolbox. Journal of Open Research Software. 2021;9(1):8. doi:10.5334/jors.251.

4. Temurshoev U, Miller RE, Bouwmeester MC. A note on the GRAS method. Economic Systems Research. 2013;25(3):361–367. doi:10.1080/09535314.2012.746645.

5. Robinson S, Cattaneo A, El-Said M. Updating and estimating a social accounting matrix using cross entropy methods. Economic Systems Research. 2001;13(1):47–64. doi:10.1080/09535310120026247.